



Strathmore University

@iLabAfrica Centre

Session 1: Introduction to Machine Learning & End-to-End ML Projects

Javier Serrano Emmanuel Bundi

Applied Machine Learning

Master in Data Science & Analytics

What this first lecture is for

- This is the **conceptual map** for the whole unit – before we look at the formal syllabus.
- Goal: agree on a shared, *production-grade* mental model of what an ML **system** (not just a model) is.
- We anchor everything in one running thread: a **thin-file credit-scoring** problem on mobile-money data.
- By the end you should be able to answer: *“what does it actually take to ship a model that keeps working?”*

Session arc

1. What ML is (and is not)
2. The ML landscape / taxonomy
3. Why ML systems fail in production
4. The end-to-end lifecycle
5. Framing & metrics
6. Data, splits & leakage
7. Baselines, evaluation, generalization
8. From model to system: MLOps + agentic outlook

Roadmap

What is Machine Learning?

The ML Landscape

Why ML Systems Fail in Production

The End-to-End ML Lifecycle

Framing & Metrics

Data, Splits & Leakage

Baselines, Evaluation & Generalization

From Model to System: MLOps & Agentic Outlook

Beyond curve fitting: a working definition

Mitchell (1997), operational form

A program learns from experience E w.r.t. a task T and performance measure P if its performance at T , measured by P , improves with E .

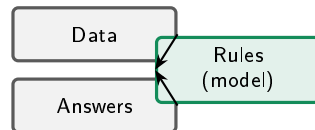
- **T**: flag a transaction as fraud
- **E**: 18 months of labelled M-Pesa-style logs
- **P**: cost-weighted recall at fixed alert budget

ML is the discipline of turning E into a function we can *deploy*, *evaluate*, and *maintain*.

Classical programming



Machine learning



ML *induces* the rules from examples instead of us coding them by hand.

When ML earns its keep – and when it does not

Use ML when

- No tractable closed-form rule (fraud patterns shift weekly).
- The environment **drifts** – a static rulebook rots.
- The problem is high-dimensional / non-linear.
- You want to **extract structure** from data (segmentation, anomaly discovery).

Rule of thumb: start with the **simplest thing that could possibly work**, then justify every added gram of ML complexity.

Prefer rules / heuristics when

- A short, auditable rule already solves it.
- Data is scarce, noisy, or unlabelled and cannot be obtained.
- Errors are catastrophic *and* must be fully explainable (some regulatory credit decisions).
- Cost of a maintained ML system $>$ value it adds.

The formal core: empirical risk minimization

We assume samples (x, y) drawn i.i.d. from an unknown distribution $\mathcal{D}$. We want a hypothesis $f \in \mathcal{H}$ minimizing the **expected risk**:

$$R(f) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [L(y, f(x))].$$

$\mathcal{D}$ is unknown, so we minimize the **empirical risk** over n training points:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))}_{\hat{R}(f)} + \lambda \Omega(f).$$

- L : loss (squared error, log-loss, cost-sensitive loss for imbalanced default data).
- Ω : regularizer controlling capacity; λ trades fit against complexity.
- Everything downstream – splits, metrics, drift – exists because $\hat{R} \neq R$. **The whole engineering problem is closing that gap and keeping it closed.**

Roadmap

What is Machine Learning?

The ML Landscape

Why ML Systems Fail in Production

The End-to-End ML Lifecycle

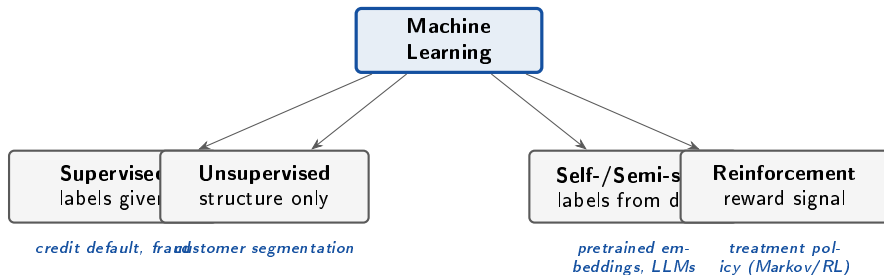
Framing & Metrics

Data, Splits & Leakage

Baselines, Evaluation & Generalization

From Model to System: MLOps & Agentic Outlook

A taxonomy you will actually use



The boundaries blur in modern systems: an LLM is self-supervised pretraining + RLHF; an agentic underwriter wraps a supervised scorer inside an RL-style decision loop.

Mapping the four paradigms to our domain

Paradigm	Signal	African applied example	Course week
Supervised	(x, y) pairs	Thin-file credit default classification on mobile-money logs	Wk 3–4
Unsupervised	x only	Merchant / borrower segmentation; KYC anomaly clusters	Wk 5
Self-supervised	pretext labels from x	Transaction embeddings; LLMs for document under-writing	Wk 10
Reinforcement	reward r_t	Sequential treatment policy via Markov / RL on clinical paths	elective / capstone

We spend most of the unit in the supervised regime because that is where deployable fintech value concentrates – then layer GenAI and agentic patterns on top.

Two orthogonal axes that decide your architecture

Batch vs. online

- **Batch:** retrain periodically on a snapshot. Simple, reproducible; lags drift.
- **Online / incremental:** update per mini-batch or per event. Needed for streaming fraud on payment rails.

These choices are *operational*, not academic: they set your latency budget, retraining cadence, and infra cost – exactly the concerns we formalize in the MLOps weeks.

Instance vs. model-based

- **Instance:** memorize, compare (k -NN). Cheap to train, costly to serve at scale.
- **Model-based:** fit parameters, generalize. The default for production scoring.

Roadmap

What is Machine Learning?

The ML Landscape

Why ML Systems Fail in Production

The End-to-End ML Lifecycle

Framing & Metrics

Data, Splits & Leakage

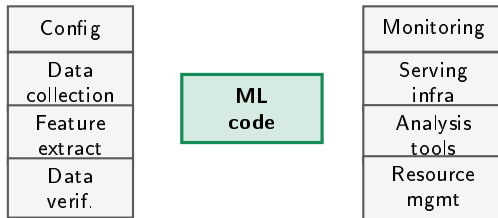
Baselines, Evaluation & Generalization

From Model to System: MLOps & Agentic Outlook

The uncomfortable truth: ML code is the small box

Sculley et al. (NeurIPS 2015), *Hidden Technical Debt in ML Systems*: the model is a **tiny fraction** of a real deployment.

- Most effort: data collection, verification, serving infra, monitoring, configuration.
- ML adds *new* debt: entanglement (CACE), feedback loops, training/serving skew, silent data dependencies.
- A great notebook is not a system.



Schematic after Sculley et al. (2015): the green core is the model; everything else is the system.

The notebook-to-production gap (our local realities)

Generic failure modes

- **Training/serving skew:** features computed differently offline vs. online.
- **Data & concept drift:**
 $\mathcal{D}_{\text{train}} \neq \mathcal{D}_{\text{serve}}$ over time.
- **Leakage:** optimistic offline metrics, collapse in production.
- **No reproducibility:** “works on my machine”.

African-context amplifiers

- **Thin-file** applicants: sparse, short credit histories.
- **Alternative data** (airtime, mobile-money) noisy and shifting.
- **Connectivity / latency** constraints at the edge.
- **Regulatory & fairness** pressure on automated credit decisions.

The model is rarely why projects fail. The *system around it* is.

Roadmap

What is Machine Learning?

The ML Landscape

Why ML Systems Fail in Production

The End-to-End ML Lifecycle

Framing & Metrics

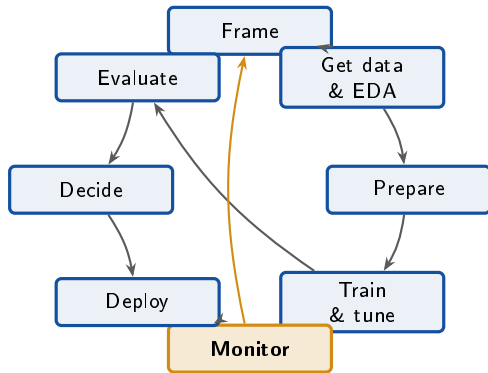
Data, Splits & Leakage

Baselines, Evaluation & Generalization

From Model to System: MLOps & Agentic Outlook

Géron's eight steps, reframed as a **loop** with monitoring as the trigger for the next turn:

1. Frame the problem
2. Get & explore the data
3. Prepare the data
4. Shortlist & train models
5. Fine-tune & select
6. Present / decide
7. Deploy
8. Monitor & maintain



It is a **cycle**: monitoring feeds re-framing and retraining.

It is a loop, not a waterfall

- Each turn of the loop is cheap to start and expensive to finish – **iterate fast on the early steps.**
- **80% of the work** lives in steps 1–3 (framing + data), not in model selection.
- A deployed model is a *liability under maintenance*, not a finished artifact: drift, retraining, and rollback are first-class.
- This unit walks the entire loop once, then has you walk it again – solo – in the capstone.

Looking ahead: Wk 2 = data prep & leakage · Wk 3–7 = modeling · Wk 11–13 = deploy, track, monitor · Wk 14–15 = capstone.

Roadmap

What is Machine Learning?

The ML Landscape

Why ML Systems Fail in Production

The End-to-End ML Lifecycle

Framing & Metrics

Data, Splits & Leakage

Baselines, Evaluation & Generalization

From Model to System: MLOps & Agentic Outlook

Step 1 is where projects are won: framing

Running case: a lender wants to extend working-capital credit to **thin-file** micro-merchants using mobile-money history.

1. **Business objective:** approve more good borrowers without raising default losses past a threshold.
2. **ML problem type:** supervised *binary classification* – $P(\text{default} \mid x)$ – with a cost-sensitive decision threshold.
3. **Current baseline:** a manual scorecard; any ML must beat it on *expected loss*, not on accuracy.
4. **Constraints:** explainability for the regulator, latency for in-app decisions, fairness across regions/genders.

If you cannot write down the objective, the metric, and the baseline, you are not ready to touch a model.

Choosing a performance measure (and why accuracy lies)

Regression metrics

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2}$$

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i|$$

RMSE punishes large errors more (sensitive to outliers / fat-tailed loss); MAE is robust. Choose by *how the business pays for errors*.

We make this rigorous in Wk 3 (SMOTE, ROC/PR, threshold selection).

Imbalanced classification

Defaults may be ~5%: a model predicting “never default” scores 95% accuracy and is useless.

- Use **precision / recall**, **PR-AUC**, ROC-AUC.
- Optimize **expected cost**, not accuracy:

$$\mathbb{E}[C] = c_{\text{FN}} \text{FN} + c_{\text{FP}} \text{FP}$$

with $c_{\text{FN}} \gg c_{\text{FP}}$ when a missed default costs far more than a declined good customer (or vice-versa).

Roadmap

What is Machine Learning?

The ML Landscape

Why ML Systems Fail in Production

The End-to-End ML Lifecycle

Framing & Metrics

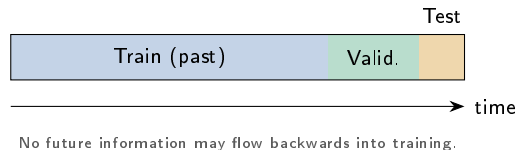
Data, Splits & Leakage

Baselines, Evaluation & Generalization

From Model to System: MLOps & Agentic Outlook

Splits: the discipline that protects your estimate of $R(f)$

- **Train / validation / test** – the test set is touched *once*, at the end.
- **Stratify** on the rare class so splits preserve the default rate.
- **Temporal split** for fintech: train on the past, test on the future – no look-ahead.
- **Data-snooping bias**: every peek at the test set leaks information and inflates your reported metric.



Data leakage: the silent killer of fintech models

Leakage = information available at training time that will *not* be available (honestly) at prediction time. It produces beautiful offline metrics and a model that collapses in production.

- **Target leakage:** a feature is a proxy for the label (e.g. “loan written off” flag used to predict default).
- **Temporal leakage:** aggregating transaction features over a window that includes the future.
- **Train/test contamination:** fitting the scaler / encoder / imputer on the full dataset before splitting.
- **Group leakage:** the same merchant appears in both train and test.

If a result looks too good, suspect leakage first. (*Full treatment + pipelines in Wk 2.*)

Roadmap

What is Machine Learning?

The ML Landscape

Why ML Systems Fail in Production

The End-to-End ML Lifecycle

Framing & Metrics

Data, Splits & Leakage

Baselines, Evaluation & Generalization

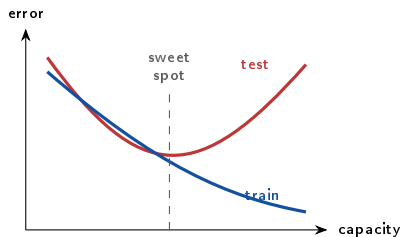
From Model to System: MLOps & Agentic Outlook

Why generalization is hard: the bias–variance decomposition

For squared loss, expected test error at a point x decomposes as:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\text{Bias}[\hat{f}(x)])^2}_{\text{too-simple model}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{too-flexible model}} + \underbrace{\sigma^2}_{\text{irreducible noise}}.$$

- **High bias** (underfit): model too rigid – train *and* test error high.
- **High variance** (overfit): model memorizes – low train, high test error.
- Regularization $\lambda \Omega(f)$, more data, and simpler $\mathcal{H}$ move you along this trade-off.
- σ^2 is the noise floor: no model beats it; chasing it is how you overfit.



Baselines and cross-validation: cheap insurance

k-fold cross-validation

Always ship a baseline first

- Majority-class / business-rule baseline sets the floor.
- A simple regularized model (logistic regression) is often 90% of the value.
- Report *lift over baseline*, never raw scores in isolation.

$$\hat{R}_{CV} = \frac{1}{k} \sum_{j=1}^k \hat{R}_{\text{val}}^{(j)}$$

- Reduces variance of the performance estimate.
- Use **stratified** / **time-series** CV to respect class balance and time order.
- Tune hyperparameters on CV folds; the test set stays sealed.

A strong baseline with no leakage beats a fancy model with leakage – every single time.

Roadmap

What is Machine Learning?

The ML Landscape

Why ML Systems Fail in Production

The End-to-End ML Lifecycle

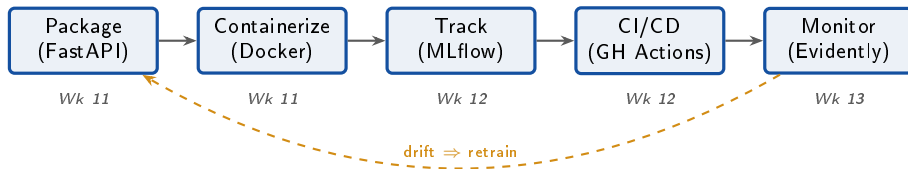
Framing & Metrics

Data, Splits & Leakage

Baselines, Evaluation & Generalization

From Model to System: MLOps & Agentic Outlook

Closing the loop: the MLOps spine of this course



This is the half of the unit that separates a data *scientist* from an ML *engineer*: a model that is served, versioned, tested, observed, and **retrained on drift**.

Where LLMs and agents fit – and where they do not

Genuinely useful

- **RAG** over policy / KYC documents for underwriting support.
- **LLM feature extraction** from unstructured statements & chat.
- **Agentic orchestration**: an agent that pulls data, scores, explains, and routes a borderline application to a human.

Modern applied ML is **hybrid**: classical models do the deciding; LLMs/agents handle language, retrieval, and orchestration around them. (*Wk 10.*)

Still the wrong tool

- Core **credit scoring** on tabular data – gradient-boosted trees still win on accuracy, latency, and auditability.
- Anything needing **calibrated probabilities** and **regulatory explainability**.
- Hard **latency / cost** budgets at the edge.

Our running thread & today's lab

The thread for the whole unit

- One domain – **thin-file mobile-money credit** – carried from framing (today) to a deployed, monitored API (capstone).
- Every week adds one ring of the lifecycle to the same problem.
- You will feel *why* each technique exists because you will have hit the wall it solves.

Lab today (1.5h)

1. Git + VS Code + isolated venv / conda.
2. Project scaffold (`src/`, `data/`, `notebooks/`).
3. Load a messy, real-world tabular dataset; honest train/test split.
4. Build an **end-to-end baseline pipeline**: preprocess → fit → evaluate vs. a majority-class baseline.

Takeaways before we open the syllabus

1. ML *induces* rules from data; the engineering job is closing the gap between empirical and expected risk – and **keeping it closed**.
2. The model is the **small green box**; the system around it is the real work.
3. The lifecycle is a **loop** driven by monitoring, not a one-shot pipeline.
4. **Framing, metrics, splits, and leakage** decide success before any model is trained.
5. Default to a strong, leak-free **baseline**; justify every added gram of complexity.
6. Modern applied ML is **hybrid**: classical cores, GenAI/agents at the edges.

Next: the formal course syllabus, assessment, and schedule.

Questions before we look at the syllabus?

Then: DSA 8401 Course Outline → Lab: build your first end-to-end baseline.